Dot Net Programming (BCA 5th Semester)

25 Most Important Short Questions & Answers (5 Marks)

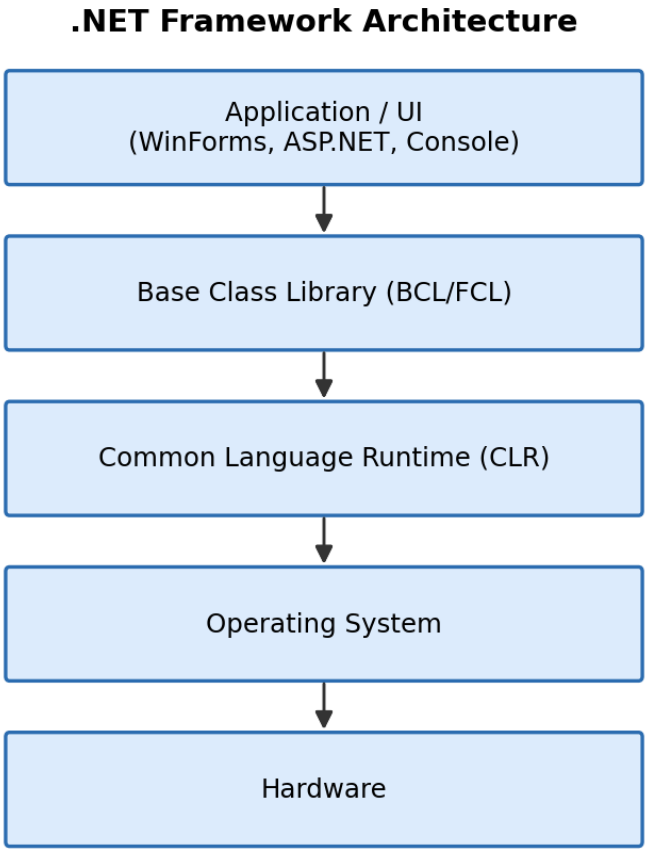
Tribhuvan University – Exam Preparation Notes

Each answer is presented in point form with diagrams/figures and C# code examples where applicable.

Unit 1: Introduction to .NET Framework

Q1. Explain the architecture of the .NET Framework with a neat diagram.

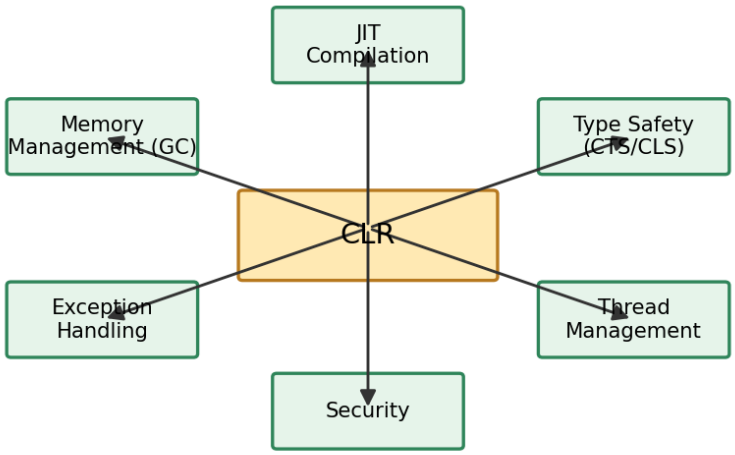
- **Definition:** .NET Framework is a software platform developed by Microsoft that provides a controlled environment (CLR) for developing and running applications.
- **Layered architecture:** it is organized in layers, each depending on the layer below it.
- **Application/Presentation layer:** user interface such as Windows Forms, ASP.NET web pages, or console apps.
- **Base Class Library (BCL/FCL):** provides ready-made classes for file I/O, collections, networking, etc.
- **Common Language Runtime (CLR):** executes the compiled (IL) code; manages memory, security and exceptions.
- **Operating System & Hardware:** CLR ultimately talks to the OS which manages the physical hardware resources.



Q2. What is CLR? Explain its major features.

- **Full form:** Common Language Runtime.
- **Definition:** CLR is the execution engine of .NET that manages the running of managed code.
- **JIT Compilation:** converts Intermediate Language (IL) code into native machine code at run time.
- **Memory Management:** automatic allocation and Garbage Collection (GC) of unused objects.
- **Exception Handling:** provides a structured mechanism (try/catch/finally) across all .NET languages.
- **Security:** enforces Code Access Security and type safety.
- **Thread Management:** supports creation and management of multiple threads.
- **Type Safety:** enforced through CTS (Common Type System) and CLS (Common Language Specification).

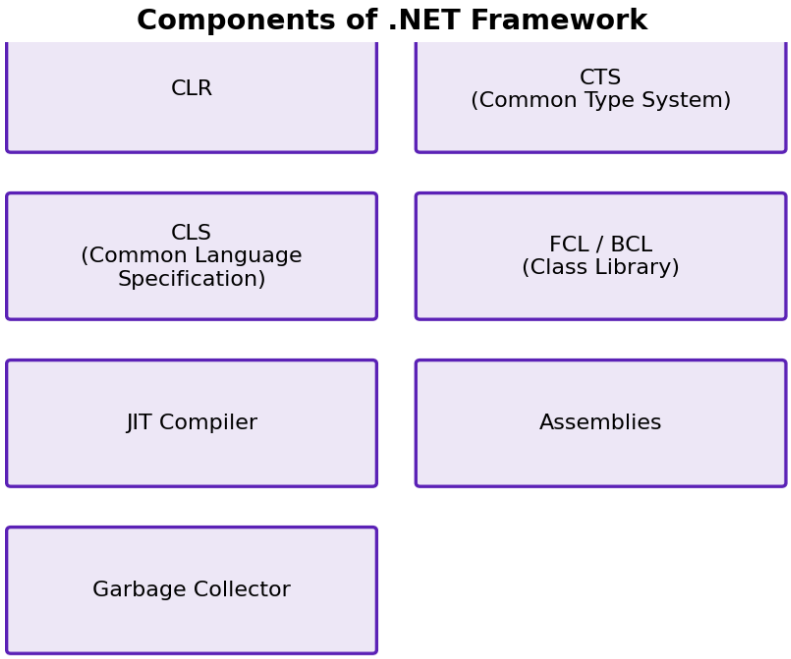
Major Features of CLR



Q3. Explain the components of the .NET Framework.

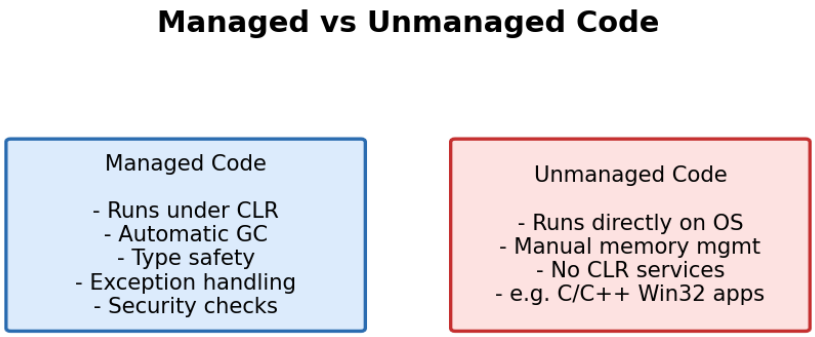
- **CLR (Common Language Runtime):** executes managed code and provides core runtime services.
- **CTS (Common Type System):** defines how types are declared, used and managed, ensuring cross-language type compatibility.
- **CLS (Common Language Specification):** a set of rules/guidelines that all .NET languages must follow for interoperability.

- **FCL / BCL (Framework/Base Class Library):** a huge collection of reusable, ready-made classes (collections, I/O, networking, etc.).
- **JIT Compiler:** translates IL code to native code just before execution.
- **Assemblies:** compiled units of deployment (.exe/.dll) containing IL code, metadata and manifest.
- **Garbage Collector (GC):** automatically reclaims memory occupied by unreachable objects.



Q4. What is managed code? How is it different from unmanaged code?

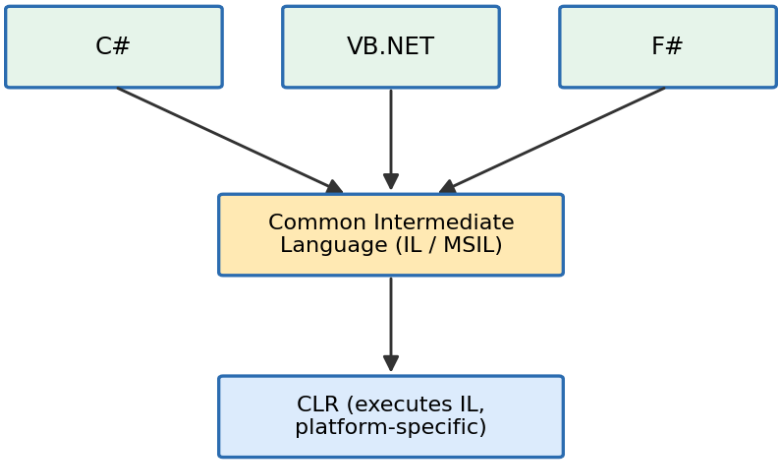
- **Managed code:** code that runs under the control of the CLR (e.g. C#, VB.NET programs).
- **Features of managed code:** gets automatic Garbage Collection, type safety, exception handling and security from CLR.
- **Unmanaged code:** code that runs directly on the operating system without CLR supervision (e.g. native C/C++ applications, Win32 API code).
- **Memory management:** managed code is automatically cleaned up by GC; unmanaged code requires the programmer to manually allocate and free memory.
- **Portability:** managed code (as IL) can run on any platform with a suitable CLR/runtime, while unmanaged code is compiled for a specific platform.



Q5. Explain why .NET provides language interoperability and platform independence.

- **Common Language Runtime (CLR):** all .NET languages (C#, VB.NET, F#) share the same CLR for execution.
- **Common Type System (CTS):** guarantees that all languages use a common set of data types, so objects created in one language can be understood in another.
- **Intermediate Language (IL):** all .NET source code compiles into a common IL/MSIL, regardless of the source language.
- **Language interoperability:** because of a shared CTS/CLS, a class written in VB.NET can be inherited or used by a class written in C#.
- **Platform independence:** IL code is not tied to any particular machine; any platform with a compatible runtime (CLR) can JIT-compile and execute it.

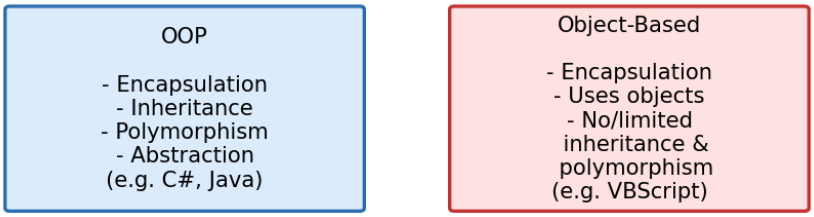
Language Interoperability via CLR/CTS/IL



Q6. Differentiate OOP and Object-Based Programming. Explain the major features of C#.

- **OOP (Object-Oriented Programming):** fully supports all 4 pillars — encapsulation, inheritance, polymorphism and abstraction (e.g. C#, Java, C++).
- **Object-Based Programming:** supports objects and encapsulation but usually lacks full inheritance and polymorphism (e.g. VBScript, early JavaScript).
- **C# feature – Simple & Modern:** clean, C-style syntax that removes complexities like pointers (in normal mode) and multiple inheritance of classes.
- **C# feature – Type-safe:** the compiler checks type compatibility, preventing many runtime errors.
- **C# feature – Component-oriented:** supports properties, events and attributes designed for building reusable components.
- **C# feature – Automatic Garbage Collection:** memory management is handled by the CLR.
- **C# feature – LINQ, delegates and events:** built-in support for querying data and event-driven programming.

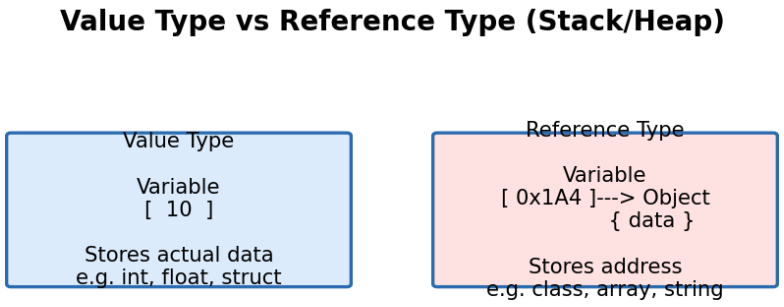
OOP vs Object-Based Programming



Unit 2: C# Programming

Q7. Differentiate value type and reference type with examples.

- **Value type:** stores the actual data directly in the variable's own memory location (usually on the Stack).
- **Reference type:** stores a reference (address) that points to the actual data stored elsewhere (on the Heap).
- **Assignment behaviour:** assigning a value type copies the data; assigning a reference type copies the address (both variables point to the same object).
- **Examples of value types:** int, float, double, char, bool, struct, enum.
- **Examples of reference types:** class, string, array, interface, delegate.

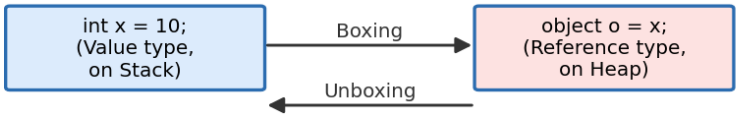


Q8. Explain boxing and unboxing with suitable examples.

- **Boxing:** the process of converting a value type into an object (reference) type; the value gets copied onto the Heap.
- **Unboxing:** the reverse process — extracting the value type from the object back into a value type variable.
- **Use:** boxing/unboxing allows value types to be treated as objects, e.g. when stored in a non-generic collection like ArrayList.
- **Performance note:** boxing/unboxing has a performance cost, so generics (List<T>) are preferred over boxing where possible.

```
int x = 10;
object o = x;      // Boxing: value type -> object
int y = (int)o;    // Unboxing: object -> value type
Console.WriteLine(y);   // Output: 10
```

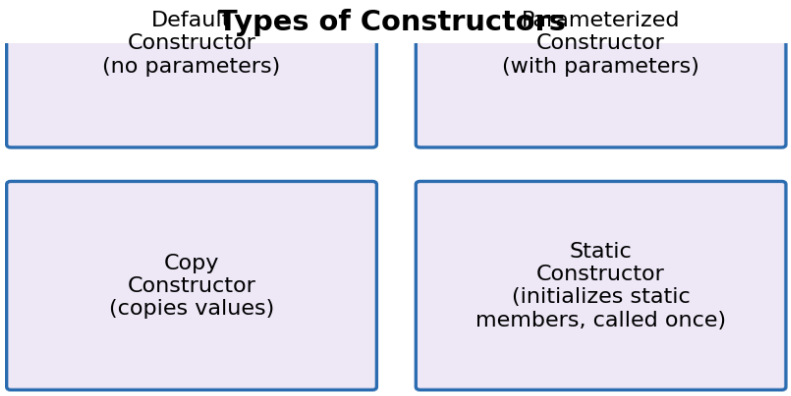
Boxing and Unboxing



Q9. Explain constructors and their different types.

- **Constructor:** a special method automatically invoked when an object of a class is created; it usually has the same name as the class and no return type.
- **Default constructor:** takes no parameters; initializes fields with default values.
- **Parameterized constructor:** accepts arguments to initialize fields with specific values at the time of object creation.
- **Copy constructor:** creates a new object by copying the values of an existing object of the same class.
- **Static constructor:** used to initialize static members; called automatically only once, before the first instance is created or any static member is referenced.

```
class Student
{
    public string Name;
    public Student() { Name = "Unknown"; }              // default
    public Student(string name) { Name = name; }         // parameterized
    public Student(Student s) { Name = s.Name; }         // copy
    static Student() { Console.WriteLine("Static ctor"); } // static
}
```

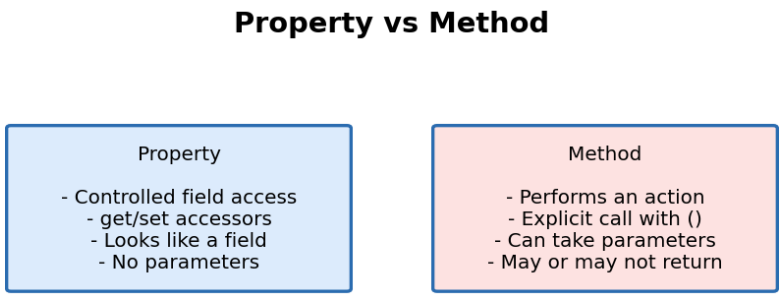


Q10. Differentiate property and method. Explain automatic property.

- **Property:** provides controlled access to a private field using get and set accessors; it is accessed like a field, without parentheses.
- **Method:** performs an action or computation; it is explicitly called using parentheses and can accept parameters.
- **Key difference:** a property looks like data but behaves like a method (encapsulating validation logic); a method always requires an explicit call.
- **Automatic property:** a shorthand syntax where the compiler automatically creates a hidden private backing field.

```
public int Age { get; set; }   // automatic property

public class Person
{
    private string name;
    public string Name            // normal property
    {
        get { return name; }
        set { name = value; }
    }
    public int Age { get; set; }  // automatic property
}
```



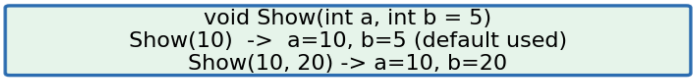
Q11. Explain optional parameters with suitable examples.

- **Definition:** a parameter that is assigned a default value in the method signature, so the caller may omit it.
- **Rule:** optional parameters must be placed after all required (non-optional) parameters in the method definition.
- **Benefit:** avoids the need for multiple overloaded methods just to handle missing arguments.

```
void Show(int a, int b = 5)
{
    Console.WriteLine(a + ", " + b);
}

Show(10);       // a = 10, b = 5  (default used)
Show(10, 20);   // a = 10, b = 20 (default overridden)
```

Optional Parameters



Q12. Explain the use of this keyword with suitable examples.

- **Definition:** this is a reference to the current instance of the class.
- **Use 1:** to access the current class's fields, properties or methods.
- **Use 2:** to resolve naming conflicts between a constructor/method parameter and a class field of the same name.
- **Use 3:** to pass the current object as an argument to another method or constructor.

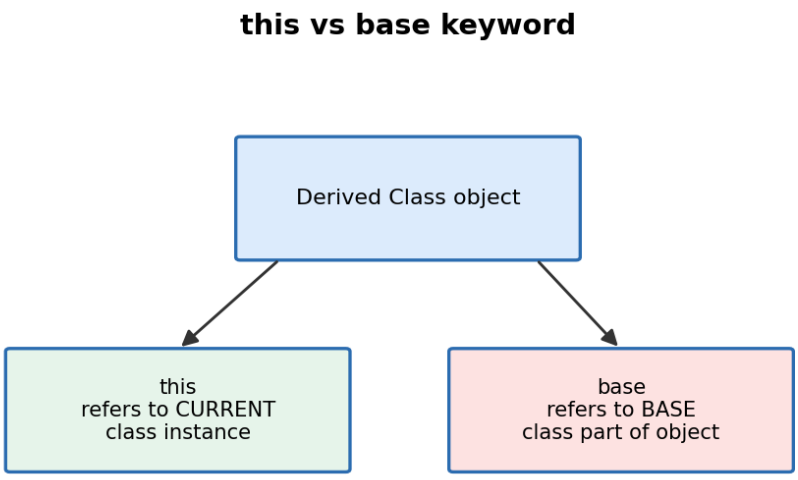
```
class Student
{
    int id;
    public Student(int id)
    {
        this.id = id;   // 'this.id' = field, 'id' = parameter
    }
}
```

Q13. Explain the use of base keyword in C#.

- **Definition:** base is a reference to the immediate base (parent) class of the current object.
- **Use 1:** to call the base class's constructor from the derived class's constructor.
- **Use 2:** to access base class members (methods/fields) that have been hidden or overridden in the derived class.

```
class Animal
{
    public void Show() { Console.WriteLine("Animal"); }
}
class Dog : Animal
{
    public void Show()
    {
```

```
        base.Show();                // calls Animal's Show()
        Console.WriteLine("Dog");
    }
}
```



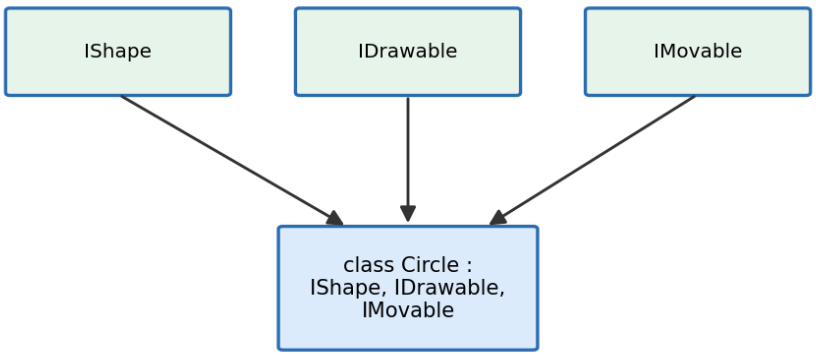
Q14. Explain interface. How is multiple inheritance achieved using interfaces?

- **Interface:** a contract that contains only method/property signatures (no implementation) which implementing classes must define.
- **Purpose:** used to achieve abstraction and to enforce a common set of behaviours across unrelated classes.
- **Multiple inheritance:** C# does not allow a class to inherit from more than one class, but a class can implement multiple interfaces, thereby achieving multiple inheritance of behaviour.
- **Implementation:** the class must provide concrete implementations of every method declared in each interface it implements.

```
interface IShape { void Draw(); }
interface IMovable { void Move(); }

class Circle : IShape, IMovable
{
    public void Draw()  { Console.WriteLine("Drawing circle"); }
    public void Move()  { Console.WriteLine("Moving circle"); }
}
```

Interfaces Enable Multiple Inheritance

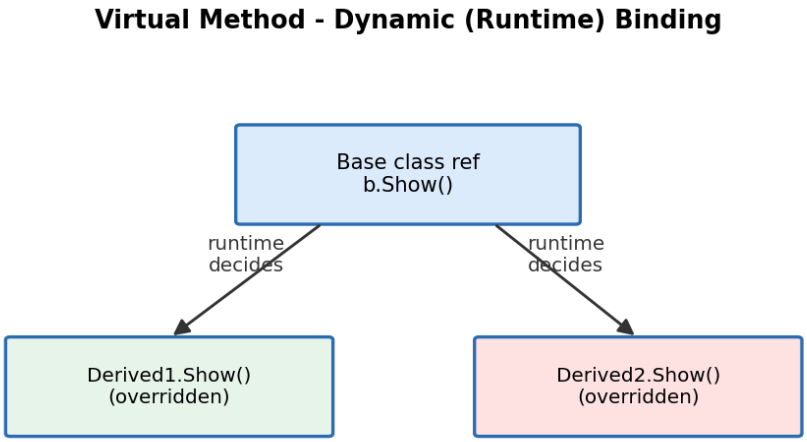


Q15. Explain virtual methods and dynamic binding.

- **Virtual method:** a method declared with the virtual keyword in the base class, which allows it to be overridden in a derived class using override.
- **Dynamic binding:** the decision of which overridden version of the method to execute is made at run time (not compile time), based on the actual object type.
- **Why needed:** enables polymorphism, so a base class reference/pointer can invoke the derived class's specific behaviour.

```
class Shape
{
    public virtual void Draw() { Console.WriteLine("Shape"); }
}
class Circle : Shape
{
    public override void Draw() { Console.WriteLine("Circle"); }
}

Shape s = new Circle();
s.Draw();     // Output: "Circle" -> decided at run time
```



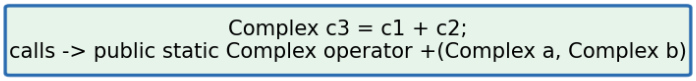
Q16. Explain operator overloading with examples.

- **Definition:** the ability to give special/additional meaning to standard operators (+, -, ==, etc.) for user-defined types (classes/structs).
- **Syntax:** achieved using the operator keyword with the static modifier.
- **Use case:** makes objects of custom types (e.g. Complex numbers, Vectors) behave naturally with familiar operators.

```
class Complex
{
    public int Real, Imag;
    public Complex(int r, int i) { Real = r; Imag = i; }

    public static Complex operator +(Complex a, Complex b)
    {
        return new Complex(a.Real + b.Real, a.Imag + b.Imag);
    }
}
Complex c3 = c1 + c2;   // calls the overloaded + operator
```

Operator Overloading

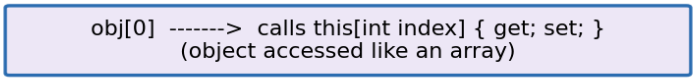


Q17. Explain indexers in C#.

- **Definition:** a special class member that allows an object to be accessed using array-like syntax obj[index].
- **Syntax:** defined using the this keyword with an index parameter, and get/set accessors, similar to a property.
- **Use case:** very useful in custom collection classes to provide simple element access.

```
class Marks
{
    private int[] arr = new int[5];
    public int this[int index]
    {
        get { return arr[index]; }
        set { arr[index] = value; }
    }
}
Marks m = new Marks();
m[0] = 90;              // uses the indexer's set
Console.WriteLine(m[0]); // uses the indexer's get
```

Indexers



Unit 3: Advanced C#

Q18. Explain delegates with suitable examples.

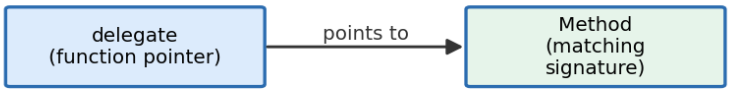
- **Definition:** a delegate is a type-safe object that holds a reference (pointer) to a method with a matching signature.
- **Purpose:** allows methods to be passed as parameters, enables callback mechanisms, and is the foundation of events.
- **Type-safe:** unlike a plain function pointer in C/C++, the compiler checks that the referenced method's signature matches the delegate's declaration.

```
delegate int Calculate(int a, int b);

int Add(int a, int b) { return a + b; }

Calculate calc = Add;
Console.WriteLine(calc(5, 3));   // Output: 8
```

Delegate: Type-safe Function Pointer



Q19. Explain generic classes and generic delegates.

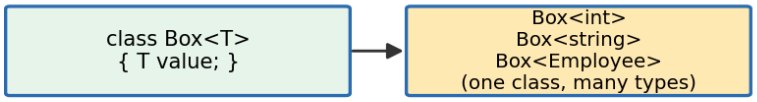
- **Generic class:** a class defined with a type parameter (e.g. <T>) so it can work with any data type without rewriting the code.
- **Benefit:** provides type safety and reusability, and avoids the performance cost of boxing/unboxing.
- **Generic delegate:** a delegate that also uses a type parameter, allowing it to reference methods of varying data types (e.g. built-in Func<T> and Action<T>).

```
class Box<T>
{
    public T Value;
    public Box(T value) { Value = value; }
}

Box<int> intBox = new Box<int>(10);
Box<string> strBox = new Box<string>("Hello");

delegate T Transformer<T>(T input);          // generic delegate
Transformer<int> square = x => x * x;
```

Generic Class - reusable with any type <T>



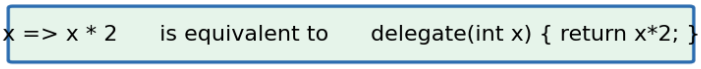
Q20. Explain lambda expressions with examples.

- **Definition:** a lambda expression is a concise, short-hand way of writing an anonymous function/method.
- **Syntax:** written using the => ("goes to") operator: (parameters) => expression/statement.
- **Common use:** extensively used with delegates, LINQ queries and event handlers to write inline logic.

```
Func<int, int> square = x => x * x;
Console.WriteLine(square(5));   // Output: 25

var evenNumbers = numbers.Where(x => x % 2 == 0);   // used in LINQ
```

Lambda Expression



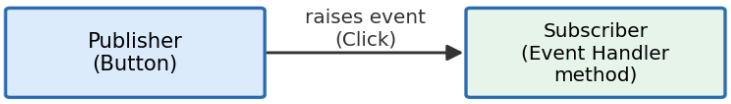
Q21. Explain events and event handling in C#.

- **Event:** a mechanism that allows an object (publisher) to notify other objects (subscribers) when something of interest happens.
- **Built on delegates:** an event is declared using the event keyword together with a delegate type.
- **Publisher-subscriber model:** the publisher raises/fires the event; any subscriber method attached to it (event handler) gets executed automatically.
- **Example scenario:** a button's Click event, or a temperature sensor's AlarmTriggered event.

```
public class Alarm
{
    public event EventHandler Triggered;
    public void Ring() { Triggered?.Invoke(this, EventArgs.Empty); }
}

Alarm a = new Alarm();
a.Triggered += (sender, e) => Console.WriteLine("Alarm ringing!");
a.Ring();   // event handler executes
```

Event & Event Handling (built on Delegates)

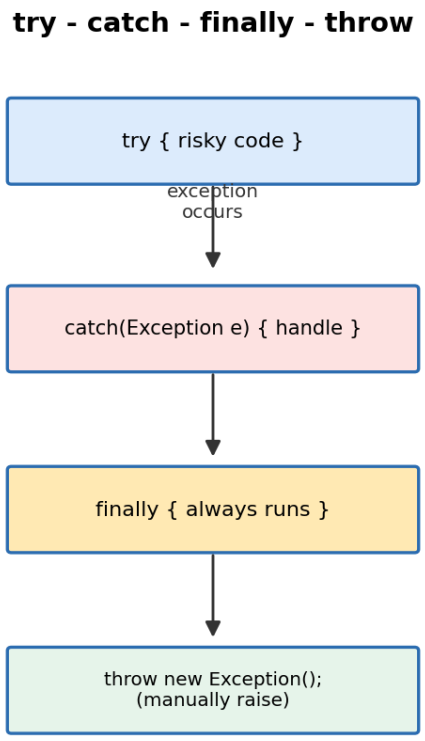


Q22. Explain exception handling in C#. Discuss try, catch, finally and throw keywords.

- **Exception handling:** a structured mechanism to detect and handle run-time errors gracefully, without crashing the program.
- **try block:** encloses the risky code that might raise an exception.

- **catch block:** catches and handles a specific (or general) exception type thrown from the try block.
- **finally block:** always executes, whether or not an exception occurred — typically used to release resources.
- **throw statement:** used to manually raise/re-raise an exception.

```
try
{
    int result = 10 / 0;
}
catch (DivideByZeroException e)
{
    Console.WriteLine("Error: " + e.Message);
}
finally
{
    Console.WriteLine("Cleanup code runs always");
}
throw new ArgumentException("Invalid input");
```



Q23. Explain finalizer and Garbage Collection in .NET.

- **Finalizer:** a special method (~ClassName()) that is automatically invoked just before an object is destroyed, used to release unmanaged resources.

- **Garbage Collector (GC):** a CLR component that automatically detects and reclaims memory occupied by objects that are no longer reachable/used by the program.
- **Benefit:** frees the programmer from manual memory management, reducing memory leaks and dangling pointer errors.

```
class Resource
{
    ~Resource()   // Finalizer
    {
        Console.WriteLine("Finalizer called, cleaning up");
    }
}
```

Finalizer and Garbage Collection

Q24. Explain LINQ query syntax. Describe Select, Where, OrderBy, and Contains operators.

- **LINQ:** Language Integrated Query — a uniform way to query data from collections, databases, XML, etc., directly within C#.
- **Select:** projects/chooses specific fields or transforms each element of the collection.
- **Where:** filters elements of a collection based on a given condition/predicate.
- **OrderBy:** sorts the elements of a collection in ascending (or descending with OrderByDescending) order.
- **Contains:** checks whether a collection contains a specified element, returning a boolean.

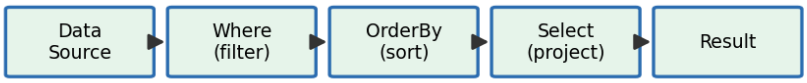
```
var students = new List<string> { "Ram", "Sita", "Hari", "Gita" };

var result = students
    .Where(s => s.Length > 3)      // filter
```

```
    .OrderBy(s => s)               // sort
    .Select(s => s.ToUpper());     // project

bool hasRam = students.Contains("Ram");   // true
```

LINQ Query Pipeline



Unit 4: ASP.NET

Q25. Explain the components of ASP.NET. Describe different validation server controls.

- **Web Forms:** the .aspx pages that define the visual/UI layout of an ASP.NET web application.
- **Server Controls:** reusable UI components (e.g. Button, TextBox, GridView) that run on the server and render HTML to the browser.
- **Code-behind:** a separate file (.aspx.cs) containing the C# logic associated with a Web Form's events and behaviour.
- **View State:** a mechanism that preserves the values of page and control properties across postbacks (round trips to the server).
- **RequiredFieldValidator:** ensures a control's value is not left empty.
- **RangeValidator:** checks that a value falls within a specified minimum and maximum range.
- **CompareValidator:** compares the value of one control to another control's value or a fixed value.
- **RegularExpressionValidator:** validates input against a defined pattern (e.g. email format).
- **CustomValidator:** allows user-defined custom validation logic.
- **ValidationSummary:** displays a summary of all validation errors on the page in one place.

ASP.NET Web Forms - Components